

Day 11 — CTEs & Subqueries

Nesting Queries: Every Type, Every Rule, With Full Worked Examples

All examples continue using the customers and orders tables from Day 10:

```
customers(customer_id, customer_name, country)
orders(order_id, customer_id, order_date, amount)
```

1. What Is a Subquery? (Theory)

A subquery is a query written *inside* another query - it runs first, and its result is used by the outer (main) query. Subqueries can appear in several different positions, and each position behaves differently:

- **In the WHERE clause** - to filter rows based on a computed value
- **In the FROM clause** - treated as a temporary, unnamed table to query from
- **In the SELECT clause** - to compute a single value per row

Key takeaway: *A subquery is just a query result being used as an input to another query - the same idea as saving a function's return value into a variable, then using that variable in the next line of Python.*

2. Subquery in WHERE

Used to filter rows in the outer query based on a value or set of values computed by the inner query.

a) Subquery returning a single value

```
-- Find all orders above the average order amount
SELECT order_id, amount
FROM orders
WHERE amount > (
    SELECT AVG(amount) FROM orders
);
```

The inner query (`SELECT AVG(amount) FROM orders`) runs once, returns a single number, and the outer query compares every row's amount against that number.

b) Subquery returning a list, used with IN

```
-- Find customers who have placed at least one order
SELECT customer_name
FROM customers
WHERE customer_id IN (
    SELECT customer_id FROM orders
);
```

Here the inner query returns a *list* of `customer_ids`, and `IN` checks whether each outer row's `customer_id` appears anywhere in that list.

c) Subquery with NOT IN (the opposite question)

```
-- Find customers who have NEVER placed an order
SELECT customer_name
FROM customers
WHERE customer_id NOT IN (
    SELECT customer_id FROM orders
);
```

Warning: if the inner query can return NULL values, NOT IN can silently return zero rows unexpectedly. It is safer to filter NULLs out inside the subquery: WHERE customer_id IS NOT NULL.

Key takeaway: Use a single-value subquery when comparing against one number (AVG, MAX, etc.); use IN/NOT IN when comparing against a list of values.

3. Subquery in FROM (Derived Table)

A subquery in the FROM clause is treated as a temporary table that only exists for the duration of this query. It must be given an alias.

```
-- Total spend per customer, then filter on that total
SELECT customer_id, total_spent
FROM (
    SELECT customer_id, SUM(amount) AS total_spent
    FROM orders
    GROUP BY customer_id
) AS customer_totals
WHERE total_spent > 1000;
```

Why not just use HAVING here? You could - GROUP BY customer_id HAVING SUM(amount) > 1000 does the same thing in this simple case. Subqueries in FROM become necessary when you need to do further work (another join, another filter) *on top of* an already-aggregated result - HAVING alone can't do that.

Key takeaway: A subquery in FROM lets you treat a query's result as if it were a real table - useful whenever one aggregation needs to feed into another calculation.

4. Subquery in SELECT

A subquery placed inside the SELECT list computes one value per row of the outer query - it must return exactly one value per row, or the query errors out.

```
-- Show each customer alongside their total number of orders
SELECT
    customer_name,
    (SELECT COUNT(*) FROM orders o WHERE o.customer_id = c.customer_id) AS order_count
FROM customers c;
```

Key takeaway: This pattern is convenient for small lookups but can be slow on large tables, since the subquery re-runs once for every outer row. A LEFT JOIN + GROUP BY (Day 9-10) usually

achieves the same result more efficiently.

5. Correlated Subqueries (Conceptual)

Definition: A correlated subquery is a subquery that references a column from the *outer* query - meaning it cannot run independently on its own. It re-executes once for every row processed by the outer query.

The SELECT-clause example above is actually a correlated subquery - notice `o.customer_id = c.customer_id` reaches out to `c`, which belongs to the outer query.

Another common example - finding each customer's most recent order:

```
SELECT o.order_id, o.customer_id, o.order_date
FROM orders o
WHERE o.order_date = (
    SELECT MAX(o2.order_date)
    FROM orders o2
    WHERE o2.customer_id = o.customer_id    -- correlated: refers to outer "o"
);
```

Regular (non-correlated) vs. correlated subquery:

Type	Behavior
Regular subquery	Runs once, independently; result is reused for every outer row
Correlated subquery	Re-runs once per outer row, using that row's value each time

Key takeaway: *Correlated subqueries are powerful for row-by-row comparisons ("latest per group," "rank within group") but can be slow on large datasets since they conceptually run once per row. On Day 12, window functions will solve many of these same problems more efficiently.*

6. CTEs (Common Table Expressions)

A CTE, written with `WITH`, gives a name to a subquery and defines it *before* the main query - making complex queries far more readable than nesting subqueries inside each other.

Basic syntax:

```
WITH customer_totals AS (
    SELECT customer_id, SUM(amount) AS total_spent
    FROM orders
    GROUP BY customer_id
)
SELECT c.customer_name, ct.total_spent
FROM customer_totals ct
JOIN customers c ON c.customer_id = ct.customer_id
WHERE ct.total_spent > 1000;
```

Compare this to the equivalent subquery-in-FROM version from Section 3 - notice the CTE version reads top-to-bottom, in the order you'd explain it out loud, instead of inside-out.

Why prefer CTEs over subqueries:

- **Readability** - named, top-down structure instead of deeply nested parentheses
- **Reusability** - the same CTE can be referenced more than once in the main query
- **Easier debugging** - you can run just the CTE's SELECT on its own to check its output before running the full query

Key takeaway: A CTE and a subquery-in-FROM often produce identical results and identical performance - the difference is almost entirely readability and maintainability, which matters enormously once queries get complex.

7. Multiple CTEs

More than one CTE can be defined in the same query, separated by commas. Later CTEs can even reference earlier ones - building a clean, step-by-step pipeline of logic.

```
WITH customer_totals AS (  
    SELECT customer_id, SUM(amount) AS total_spent  
    FROM orders  
    GROUP BY customer_id  
)  
high_value_customers AS (  
    SELECT customer_id  
    FROM customer_totals  
    WHERE total_spent > 1000  
)  
SELECT c.customer_name, ct.total_spent  
FROM high_value_customers hvc  
JOIN customer_totals ct ON ct.customer_id = hvc.customer_id  
JOIN customers c ON c.customer_id = hvc.customer_id;
```

Breaking this down step by step:

- `customer_totals` - Step 1: aggregate raw orders into one total per customer
- `high_value_customers` - Step 2: filter that result down to just the `customer_ids` that qualify
- Final SELECT - Step 3: join everything back together for the final, readable output

Key takeaway: Multiple CTEs let you build a query the same way you'd build a Python script - one clearly-named step at a time - instead of one giant, tangled expression. This is exactly the same instinct as Day 3's "modular project structure," applied to SQL.

8. Recursive CTE (Awareness Only)

A recursive CTE is a CTE that *references itself*, allowing it to repeatedly build on its own previous result. It is used for hierarchical or graph-like data - for example, an employee-manager org chart, or a category-subcategory product tree.

Structure (conceptual):

- **Anchor member** - the starting point (e.g., the top-level manager with no boss)
- **UNION ALL** - combines the anchor with the recursive step
- **Recursive member** - a query that refers back to the CTE's own name, finding the "next level down" each time it repeats

```
-- Conceptual example: finding all employees under a manager, at any depth
WITH RECURSIVE employee_chain AS (
  -- Anchor: the top manager
  SELECT employee_id, manager_id, employee_name
  FROM employees
  WHERE manager_id IS NULL

  UNION ALL

  -- Recursive step: find employees reporting to anyone already in employee_chain
  SELECT e.employee_id, e.manager_id, e.employee_name
  FROM employees e
  JOIN employee_chain ec ON e.manager_id = ec.employee_id
)
SELECT * FROM employee_chain;
```

Key takeaway (awareness level only): *You don't need to master recursive CTEs right now - just recognize the pattern and know it exists for hierarchical data. It's rare in day-to-day data engineering work but does appear in org charts, bill-of-materials data, and folder/category trees.*

Day 11 Summary — CTEs & Subqueries

Concept	One-Line Recap
Subquery in WHERE	Filters outer rows using a computed value or list from an inner query
Subquery in FROM	Treats a query's result as a temporary table to query further
Subquery in SELECT	Computes one value per outer row - can be slow on large tables
Correlated Subquery	References the outer query's row - re-runs once per outer row
CTE (WITH)	Names a subquery upfront for a readable, top-down query structure
Multiple CTEs	Chains named steps together, like a modular pipeline
Recursive CTE	A CTE that references itself, for hierarchical/tree-shaped data

Overall takeaway: *Subqueries and CTEs solve the same underlying problem - using one query's result inside another - but CTEs almost always win on readability once logic gets more than one step deep. Default to CTEs for anything non-trivial, and reach for a correlated subquery only when you specifically need a row-by-row comparison.*